

Exemple, ExempleCollection (v1)

Fabrice.Kordon@lip6.fr



Objectif de l'exemple

2

Montrer de la manipulation simple

L'archi-classique ArrayList

► Implémentation de l'interface Collection

► <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>

► «java 8 arraylist» dans Google

Quelques méthodes...

```
boolean add(E e)
void add (int index, E element)
boolean addAll(Collection<? extends E> c)
boolean addAll(int index, Collection<? extends E> c)
void clear()
E remove (int index)
boolean removeAll(Collection<?> c)
boolean removeIf(Predicate<? super E> filter)
void clone()
boolean contains(Object o)
E get(int index)
boolean isEmpty()
int size()
Object[] toArray()
```


La classe Etudiant

3

```
import java.util.ArrayList;

class Etudiant {

    private final String nom;
    private final String prenom;
    private final int numero;

    Etudiant (String n, String p, int id) {
        nom = n;
        prenom = p;
        numero = id;
    }

    public boolean estPair() {
        return numero % 2 == 0;
    }

    public String toString () {
        return nom + " " + prenom + "(" + numero + ")";
    }

}
```


La classe ExempleCollections1

4

```
public class ExempleCollections1 {
    static int maxEtudiants;

    public static void main(String []args) {
        try {
            maxEtudiants = Integer.parseInt(args[0]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.err.println ("Arguments requis, nombre d'étudiants.");
            return;
        }

        ArrayList<Etudiant> mesEtudiants= new ArrayList<Etudiant>();

        for (int i = 1 ; i <= maxEtudiants ; i++) {
            boolean b = mesEtudiants.add(new Etudiant("nom_" + i, "prénom_" + i, i));
        }
        System.out.println ("La collection contient " + mesEtudiants.size() + " étudiant(s).");
        mesEtudiants.remove(0);
        System.out.println ("Voici l'état de la collection après suppression du premier...");
        mesEtudiants.forEach((e) -> {System.out.println ("    " + e);});
        mesEtudiants.removeIf((e) -> {return e.estPair();});
        System.out.println ("Voici l'état de la collection après suppression des pairs...");
        mesEtudiants.forEach((e) -> {System.out.println ("    " + e);});
    }
}
```


Une exécution

5

```
$ java ExempleCollections1 13
La collection contient 13 étudiant(s).
Voici l'état de la collection après suppression du premier...
  nom_2 prénom_2(2)
  nom_3 prénom_3(3)
  nom_4 prénom_4(4)
  nom_5 prénom_5(5)
  nom_6 prénom_6(6)
  nom_7 prénom_7(7)
  nom_8 prénom_8(8)
  nom_9 prénom_9(9)
  nom_10 prénom_10(10)
  nom_11 prénom_11(11)
  nom_12 prénom_12(12)
  nom_13 prénom_13(13)
Voici l'état de la collection après suppression des pairs...
  nom_3 prénom_3(3)
  nom_5 prénom_5(5)
  nom_7 prénom_7(7)
  nom_9 prénom_9(9)
  nom_11 prénom_11(11)
  nom_13 prénom_13(13)
```


Discussion...

6

On passait une portion de code...

```
mesEtudiants.forEach((e) -> {System.out.println (" " + e);});  
mesEtudiants.removeIf((e) -> {return e.estPair();});
```

Peut-on passer une fonction?

 On a aimé les pointeurs de fonctions en C



Variante de la classe Etudiant

7

```
class Etudiant {  
  
    private final String nom;  
    private final String prenom;  
    private final int numero;  
  
    Etudiant (String n, String p, int id) {  
        nom = n;  
        prenom = p;  
        numero = id;  
    }  
  
    public boolean estPair() {  
        return numero % 2 == 0;  
    }  
  
    public String toString () {  
        return nom + " " + prenom + "(" + numero + ")";  
    }  
  
    static public void affiche (Etudiant e) {  
        System.out.println(" " + e);  
    }  
}
```


Variante de la «classe principale»

8

```
public class ExempleCollections1bis {

    static int maxEtudiants;

    public static void main(String []args) {
        try {
            maxEtudiants = Integer.parseInt(args[0]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.err.println ("Arguments requis, nombre d'étudiants.");
            return;
        }

        ArrayList<Etudiant> mesEtudiants= new ArrayList<Etudiant>();

        for (int i = 1 ; i <= maxEtudiants ; i++) {
            boolean b = mesEtudiants.add(new Etudiant("nom_" + i, "prénom_" + i, i));
        }
        System.out.println ("La collection contient " + mesEtudiants.size() + " étudiant(s).");
        mesEtudiants.remove(0);
        System.out.println ("Voici l'état de la collection après suppression du premier...");
        mesEtudiants.forEach((e) -> Etudiant.affiche(e)); // l'appel modifié
        mesEtudiants.removeIf((e) -> {return e.estPair();});
        System.out.println ("Voici l'état de la collection après suppression des pairs...");
        mesEtudiants.forEach((e) -> Etudiant.affiche(e)); // l'appel modifié
    }
}
```


Variante de la «classe principale»

8

```
public class ExempleCollections1bis {
```

```
    static int maxEtudiants;
```

```
    public static void main(String []args)
```

```
    {
```

```
        maxEtudiants = Integer.parseInt(args[0]);
```

```
    }
```

```
    catch (ArrayIndexOutOfBoundsException e) {
```

```
        System.err.println ("Arguments requis, nombre d'étudiants.");
```

```
        return;
```

```
    }
```

```
    ArrayList<Etudiant> mesEtudiants= new ArrayList<Etudiant>();
```

```
    for (int i = 1 ; i <= maxEtudiants ; i++) {
```

```
        boolean b = mesEtudiants.add(new Etudiant("nom_" + i, "prénom_" + i, i));
```

```
    }
```

```
    System.out.println ("La collection contient " + mesEtudiants.size() + " étudiant(s).");
```

```
    mesEtudiants.remove(0);
```

```
    System.out.println ("Voici l'état de la collection après suppression du premier...");
```

```
    mesEtudiants.forEach((e) -> Etudiant.affiche(e)); // l'appel modifié
```

```
    mesEtudiants.removeIf((e) -> {return e.estPair();});
```

```
    System.out.println ("Voici l'état de la collection après suppression des pairs...");
```

```
    mesEtudiants.forEach((e) -> Etudiant.affiche(e)); // l'appel modifié
```

```
    }
```

```
}
```

Exécution identique

et ici, sémantique similaire au passage d'un pointeur de fonction